

UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS Y COMPUTACIÓN



TITULO

Arquitecturas de Bases de Datos: Centralizada, Cliente-Servidor y Distribuida

CURSO : Base de Datos II

DOCENTE : ING.FERNANDEZ BEJARANO RAULENRIQUE

ESTUDIANTE : CONDOR DE LA CRUZ ARMANDO

CICLO : V

HUANCAYO – PERU 2025

Arquitecturas de Bases de Datos: Centralizada, Cliente-Servidor y Distribuida

Esquema conceptual previo (analogía inicial)

Analogía del restaurante:

- *Centralizada*: Un único cocinero en una cocina pequeña. Todos los comensales piden directamente a él. Si se satura o la cocina falla, nadie come.
- *Cliente-Servidor*: Varios camareros toman pedidos y los llevan a una cocina central con varios cocineros. Los camareros son clientes; la cocina, el servidor.
- *Distribuida*: Varias cocinas en diferentes sedes, cada una con sus cocineros y cámaras. Los pedidos se reparten según la ubicación del comensal, pero todas las cocinas comparten el menú y se sincronizan entre sí.

1. Arquitectura Centralizada

Definición

Todos los datos residen en un único sistema computacional (un servidor o mainframe). Los usuarios acceden directamente a través de terminales tontas o aplicaciones conectadas a ese nodo central.

Características técnicas

- Un solo punto de control y almacenamiento.
- El procesamiento de consultas ocurre íntegramente en el servidor central.
- Baja latencia interna (sin redes complejas entre nodos).
- Escalamiento vertical (mejorar hardware del servidor).

Ventajas

- Simplicidad de administración.
- Alta consistencia de datos (transacciones ACID fáciles de garantizar).
- Menor costo de hardware y software inicial.

Desventajas

- Punto único de fallo.
- Cuello de botella al crecer el número de usuarios.
- Dificultad para atender ubicaciones geográficas dispersas.

Ejemplo con DBMS actual

- **SQLite** en aplicaciones embebidas (un solo archivo como base de datos).
- **Oracle** en modo standalone para pequeñas empresas.
- **Microsoft Access** en entornos de escritorio.

2. Arquitectura Cliente-Servidor

Definición

Separación de roles: el servidor aloja el motor de base de datos y gestiona los datos; los clientes (aplicaciones, PCs, móviles) realizan peticiones mediante consultas (SQL, procedimientos remotos).

Características técnicas

- Comunicación mediante red (TCP/IP, protocolos propietarios como TDS o OCI).
- El servidor procesa las consultas y devuelve solo los resultados.
- Los clientes pueden tener lógica de presentación y de negocio.

Ventajas

- Escalabilidad horizontal moderada (añadir clientes sin afectar drásticamente).
- Mejor seguridad (control de acceso centralizado en el servidor).
- Permite interfaces gráficas ricas en los clientes.

Desventajas

- El servidor sigue siendo un punto crítico (aunque con redundancia posible).
- Sobrecarga de red por las peticiones constantes.
- Coste de licencias y administración más alto que el centralizado.

Ejemplo con DBMS actual

- **PostgreSQL** con clientes psql, pgAdmin o aplicaciones web.
- **MySQL** usado en arquitecturas LAMP.
- **Microsoft SQL Server** con aplicaciones .NET.

3. Arquitectura Distribuida

Definición

Los datos están fragmentados y/o replicados en múltiples nodos físicamente separados, pero se presentan al usuario como una única base de datos lógica. Puede ser homogénea (mismo DBMS en todos los nodos) o heterogénea.

Características técnicas

- Fragmentación horizontal (por filas) o vertical (por columnas).
- Replicación síncrona o asíncrona.
- Control de concurrencia distribuido (protocolos de consenso como Paxos o Raft).
- Transacciones distribuidas (2PC - two-phase commit).

Ventajas

- Alta disponibilidad y tolerancia a fallos.
- Escalabilidad horizontal casi ilimitada.
- Baja latencia para usuarios geográficamente dispersos (datos locales).

Desventajas

- Complejidad de diseño y administración.
- Mayor latencia en operaciones que requieren consistencia global.
- Dificultad para garantizar ACID en escenarios con particiones de red (teorema CAP).

Ejemplo con DBMS actual

- **Cassandra** (Apache) – para alta escritura y disponibilidad.
- **CockroachDB** – SQL distribuido con transacciones ACID.
- **Google Spanner** – base de datos distribuida globalmente.
- **MongoDB** en modo cluster con sharding.

Comparativa técnica resumida

Característica	Centralizada	Cliente-Servidor	Distribuida
Escalabilidad	Vertical limitada	Vertical / horizontal moderada	Horizontal ilimitada
Tolerancia a fallos	Muy baja	Media (con réplicas)	Alta
Consistencia de datos	Fuerte (ACID total)	Fuerte (ACID)	Eventual o fuerte (costosa)
Complejidad de gestión	Baja	Media	Alta
Coste operativo	Bajo inicial	Medio	Alto (hardware+software)
Latencia para usuarios remotos	Alta	Media	Baja (si hay nodo cercano)

Recomendación para un sistema de gestión académica

Arquitectura más adecuada: Cliente-Servidor con réplicas de solo lectura.

Justificación técnica

- Una universidad tiene una carga moderada de usuarios (estudiantes, profesores, administrativos) y transacciones (inscripciones, notas, pagos).
- Se requiere alta consistencia en notas y registros académicos (evitar lecturas inconsistentes).
- La geolocalización suele ser en uno o pocos campus, por lo que no justifica una distribución completa.
- El presupuesto y la capacidad técnica suelen ser limitados en entornos académicos.

Topología sugerida

- **Servidor principal** (PostgreSQL o SQL Server) para escrituras y consistencia fuerte.
- **Réplicas en caliente** (streaming replication) para lecturas de consultas de horarios, listados, etc.
- **Clientes** aplicaciones web (acceso desde navegadores) y móviles.

¿Por qué no distribuidas?

- Complejidad operativa excesiva para un equipo TI universitario promedio.
- Las transacciones distribuidas son lentas y pueden causar conflictos en inscripciones masivas.
- El teorema CAP obligaría a sacrificar consistencia o disponibilidad en momentos de red inestable.

¿Por qué no centralizada pura?

- Riesgo alto de fallo único (pérdida de calificaciones en periodo de exámenes sería catastrófico).
- No permite dar servicio a sedes remotas con baja latencia.

Ejemplo de implementación recomendada

- **DBMS:** PostgreSQL 15+ con replicación síncrona para las tablas críticas (estudiantes, notas, matrículas) y asíncrona para las de solo consulta (historial académico).
- **Middlewares de conexión:** PgBouncer para pool de conexiones.
- **Cliente:** Aplicación web en PHP, Python (Django) o Node.js que conecta al servidor principal para escrituras y a las réplicas para lecturas.

Conclusión para el informe del estudiante

El alumno debe mostrar en su organizador visual (Canvas/Genially) una tabla comparativa como la anterior, ejemplos concretos de DBMS y una justificación basada en el **teorema CAP** y en los **requisitos no funcionales** del sistema académico: consistencia sobre disponibilidad, tolerancia a fallos moderada y complejidad controlada.